รายงานสัปดาห์ที 12 (Advanced)
Agentic Communication Workflows

สรุปเนื้อหา การแก้ข้อบกพร่อง 3 จุด และการตรวจสอบความถูกต้อง
โปรเจกต์: agentic-communication-automator

---

1. ข้อจำกัดของรายงานฉบับนี้

ข้อจำกัดสำคัญ: แล็บนีส่งอีเมลจริงผ่านบัญชี Gmail ซึงต้องใช้ App Password ส่วนตัว ผู้ช่วย AI ไม่สามารถกรอกรหัสผ่านหรือส่งอีเมลแทนได้ เพราะเป็นการกระทำทีส่งผลออกไปภายนอกและต้องใช้ข้อมูลลับของเจ้าของบัญชี รายงานนีจึงครอบคลุมการเตรียมโค้ดและการตรวจสอบความถูกต้องเท่าทีทำได้โดยไม่ใช้บัญชีจริง ส่วนภาพ screenshot อีเมลทีโจทย์กำหนด ผู้เรียนต้องรันเองหลังตังค่าไฟล์ .env

2. ภาพรวมและผลลัพธ์การเรียนรู้

แทร็ก Advanced ต่อยอดจาก Basic โดยเปลียนจากการเขียนโค้ดสังส่งอีเมลทีละคำสังใน `main.py` มาเป็น agent ทีอ่านแผนงานจากไฟล์ `comm_pipeline_config.json` แล้วทำตามทีละ task
ซึงเป็นแนวคิดเดียวกับแล็บ Advanced สัปดาห์ที 8 10 และ 11

- ออกแบบ pipeline งานสือสารแบบ config-driven ทีเพิม/ลด/สลับงานได้โดยไม่ต้องแก้โค้ด
- ใช้เทมเพลตสำหรับหัวข้อและเนือหาอีเมล เติมค่าจริงตอนรัน เช่น `"Daily Sales Report - {report_date}"`
- ประมวลผลกล่องจดหมายขาเข้าตามเงือนไข (ยังไม่อ่าน / ผู้ส่ง / คำในหัวข้อ)
- ตอบกลับอัตโนมัติเมือพบอีเมลทีเข้าเงือนไข

3. โครงสร้าง pipeline ใน config

#	Task	สิงทีทำ
1	send_email	ส่งรายงานยอดขายประจำวัน พร้อมแนบ sales_report.pdf และ summary_data.xlsx
2	send_email	ส่งอีเมลจำลองคำขอข้อมูลถึงตัวเอง เพือใช้ทดสอบ task ถัดไป
3	process_inbox	ค้นหาอีเมลทียังไม่อ่านจากผู้ส่งทีกำหนด แล้วประมวลผล/ตอบกลับอัตโนมัติ

สังเกตว่า task ที 3 ต้องทำหลัง task ที 2 เพราะต้องรออีเมลจาก task ที 2 เข้ากล่องจดหมายก่อน ลำดับของ task ใน config จึงมีความสำคัญ

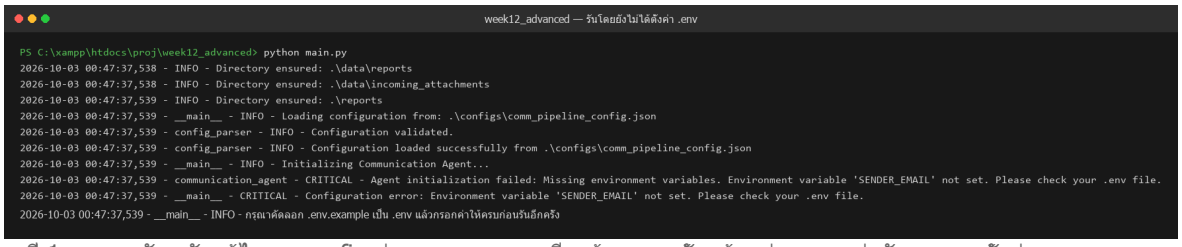
4. ข้อบกพร่องทีพบและการแก้ไข

#	ไฟล์	อาการ	การแก้ไข
1	main.py	เมือยังไม่ได้ตังค่า .env โปรแกรมโยน traceback ดิบออกมาทังก้อน ผู้ใช้อ่านไม่รู้ว่าต้องทำอะไรต่อ ซึงต่างจากเวอร์ชัน BASIC ทีแจ้งข้อความชัดเจน	แยกการดัก ValueError และ FileNotFoundError ออกจาก Exception ทัวไป พร้อมข้อความบอกวิธีแก้ว่าให้คัดลอก .env.example เป็น .env

2	utils.py	ข้อความ log ถูกพิมพ์ซ้ำ 2 บรรทัด	เพิ่ม logger.propagate = False
3	(ไฟล์ข้อมูล)	config อ้างถึงไฟล์แนบ data/reports/sales_report.pdf และ summary_data.xlsx ที่ไม่มีอยู่จริงใน repo ทำให้ task แรกจะล้มเหลวแม้ใส่ credential ถูกต้องแล้ว	สร้างไฟล์แนบตัวอย่างทั้งสองไฟล์ให้ เพื่อให้ pipeline ทำงานได้ครบทันทีที่ตั้งค่า .env เสร็จ

5. การตรวจสอบความถูกต้องโดยไม่ใช้บัญชีจริง

- Syntax check: ทุกไฟล์ใน `src/` และ `main.py` คอมไพล์ผ่าน ไม่มี syntax error
- Config validation: agent โหลดและตรวจสอบไฟล์ config ผ่านเรียบร้อย (เห็นข้อความ `Configuration validated.` ในภาพ)
- การจัดการเมื่อขาด credential: หลังแก้บักที่ 1 แล้ว แจ้งข้อความชัดเจนแทน traceback ดิบ



ภาพที่ 1: ผลการรันหลังแก้ไข — config ผ่านการตรวจสอบเรียบร้อย จากนั้นแจ้งอย่างสุภาพว่ายังขาดการตั้งค่า `.env` พร้อมบอกวิธีแก้ แทนที่จะโยน traceback ดิบแบบเดิม

6. ขั้นตอนที่ผู้เรียนต้องทำเองเพื่อส่งงาน

ขั้น	สิ่งที่ต้องทำ
1	คัดลอก .env.example เป็น .env แล้วกรอก SENDER_EMAIL และ App Password 16 หลัก
2	แก้ configs/comm_pipeline_config.json เปลี่ยนอีเมลตัวอย่างเป็นอีเมลจริงของตัวเอง
3	แก้เนื้อหาเทมเพลตอีเมลให้มี ชื่อ-นามสกุล และรหัสนักศึกษา ตามที่โจทย์กำหนด
4	รัน python main.py แล้วจับภาพกล่องจดหมายที่ได้รับอีเมล

คำถาม-คำตอบสำคัญ

SMTP กับ IMAP ต่างกันอย่างไร?

SMTP (Simple Mail Transfer Protocol) ใช้สำหรับ ส่ง อีเมล ทำงานผ่าน `smtplib` โดยเชื่อมต่อไปที่เซิร์ฟเวอร์ (เช่น smtp.gmail.com พอร์ต 587) เรียก `starttls()` เพื่อเข้ารหัสการเชื่อมต่อ แล้วจึง `login()` และส่งข้อความ

IMAP (Internet Message Access Protocol) ใช้สำหรับ อ่าน อีเมลบนเซิร์ฟเวอร์ ผ่าน `imaplib` โดยเชื่อมต่อแบบ SSL (imap.gmail.com พอร์ต 993) เลือกกล่องจดหมาย ค้นหาด้วยเงื่อนไข เช่น UNSEEN หรือ FROM แล้วดึงข้อความมาแยกส่วนด้วยโมดูล `email`

ทำไม Gmail ถึงต้องใช้ App Password แทนรหัสผ่านปกติ?

Gmail ปิดการให้โปรแกรมภายนอกล็อกอินด้วยรหัสผ่านปกติไปแล้ว เพื่อความปลอดภัย จึงต้องใช้ App Password ซึ่งเป็นรหัส 16 หลักที่สร้างแยกเฉพาะสำหรับแต่ละแอป ข้อดีคือถ้ารหัสนี้รั่วไหล เจ้าของบัญชีเพิกถอนเฉพาะรหัสนั้นได้โดยไม่กระทบรหัสผ่านหลัก และรหัสนี้ใช้ล็อกอินหน้าเว็บไม่ได้ เงื่อนไขคือต้องเปิด 2-Step Verification ก่อนจึงจะสร้างได้

ทำไมต้องเก็บ credential ไว้ในไฟล์ .env แทนที่จะเขียนในโค้ด?

เพราะถ้าเขียนรหัสผ่านไว้ในโค้ดโดยตรง เมื่อ push ขึ้น GitHub รหัสจะถูกเปิดเผยต่อสาธารณะทันที และยังคงอยู่ในประวัติ commit แม้จะลบออกภายหลังแล้วก็ตาม การเก็บไว้ในไฟล์ `.env` ที่ถูกระบุไว้ใน `.gitignore` ทำให้โค้ดแชร์ได้โดยไม่พารหัสผ่านไปด้วย ส่วนไฟล์ `.env.example` ที่ commit ขึ้นไปจะมีแต่ชื่อตัวแปรและคำอธิบาย ไม่มีค่าจริง เพื่อให้คนอื่นรู้ว่าต้องตั้งค่าอะไรบ้าง

ทำไม SMS ถึงมักส่งไม่สำเร็จ?

เทคนิค email-to-SMS gateway ที่ใช้ในแล็บนี้ คือการส่งอีเมลไปยังที่อยู่รูปแบบ ■■■■■■■■@gateway-■■■■■■■■ เช่น `1234567890@txt.att.net`
ซึ่งเป็นบริการที่ค่ายมือถือในสหรัฐอเมริกาเปิดให้ ค่ายในไทยส่วนใหญ่ไม่มีบริการนี้ จึงส่งไม่ถึงปลายทาง ซึ่งโจทย์ก็ระบุไว้แล้วว่าหากส่ง SMS ไม่ได้ไม่เป็นไร หากต้องการส่ง SMS จริงต้องใช้บริการอย่าง Twilio ที่มี API โดยตรง

บทสรุป

แล็บสัปดาห์นี้ปิดท้ายชุดงานอัตโนมัติด้วยการสื่อสารออกไปภายนอก
ซึ่งต่างจากแล็บก่อนหน้าที่ทำงานกับไฟล์ในเครื่องเท่านั้น
ความต่างที่สำคัญคือเรื่องความปลอดภัยของข้อมูลลับ เพราะต้องใช้รหัสผ่านจริงในการเชื่อมต่อ
แนวปฏิบัติที่ได้เรียนรู้คือการแยก credential ออกจากโค้ดด้วยไฟล์ `.env` การใช้ App Password แทนรหัสผ่านหลัก และการ commit เฉพาะไฟล์ `.env.example` ที่ไม่มีค่าจริง
ซึ่งเป็นมาตรฐานที่ใช้กันในงานจริงทุกโปรเจกต์ที่ต้องเชื่อมต่อบริการภายนอก

เมื่อเทียบกับแท็ก Basic แล้ว แท็ก Advanced แสดงให้เห็นว่าเมื่องานสื่อสารมีหลายขั้นตอน (ส่งรายงาน → รออีเมลเข้า → ประมวลผล → ตอบกลับ) การย้ายลำดับงานออกไปไว้ในไฟล์ config ทำให้ปรับเปลี่ยนกระบวนการได้โดยไม่ต้องแก้โค้ด ซึ่งเป็นแนวคิดเดียวกับที่ใช้มาตั้งแต่สัปดาห์ที่ 8

อ้างอิง: เอกสารประกอบการสอน "Week12_LabX ADV: Agentic Communication Workflows" และซอร์สโค้ดจาก GitHub ของผู้สอน

หมายเหตุ: ภาพในรายงานสร้างจาก output จริงที่ได้จากการรันบนเครื่อง (ไม่ใช่ภาพจับหน้าจอ OS โดยตรง)